RAPPORT DE TRAVAUX PRATIQUES

# Développement d'une Application Monolithique

# Spring Boot

**Réalisé par :** Chaymae HICHAMI-ALAOUI

**Encadrant :** Pr. BE ELBAGHAZAOUI

# 1. Introduction et Objectifs

Ce rapport présente la conception et le développement d'une **application monolithique de gestion de produits** en utilisant le framework **Spring Boot**. L'objectif était de mettre en place une **API REST CRUD complète** en respectant une architecture logicielle rigoureuse, développée sous **IntelliJ IDEA** et connectée à une base de données **PostgreSQL**.

Le projet couvre l'intégralité du cycle de développement : initialisation du projet via Spring Initializr, structuration en couches, implémentation de la logique métier, exposition des endpoints REST, puis validation fonctionnelle via Postman. Une extension a également été réalisée pour enrichir le modèle de données.

# 2. Architecture Technique

Le projet suit une **architecture en couches (Layered Architecture)**, principe fondamental du développement Spring Boot. Chaque couche a une responsabilité précise et ne communique qu'avec la couche adjacente, garantissant un code **maintenable, testable et découplé**.

| Couche | Composant | Responsabilité |
|---|---|---|
| **Modèle (Entity)** | `Product.java` | Définition de l'entité avec ses attributs (id, name, description, price, stock) et contraintes de validation |
| **Persistance (Repository)** | `ProductRepository.java` | Interface Spring Data JPA — génère automatiquement les requêtes SQL vers PostgreSQL |
| **Service** | `ProductService.java` | Logique métier centralisée — gestion des transactions |
| **Contrôleur (REST)** | `ProductController.java` | Exposition des endpoints REST — traitement des requêtes HTTP entrantes |

**Principe clé :** Cette séparation en couches garantit que chaque composant a une seule responsabilité (SRP — Single Responsibility Principle), facilitant la maintenance, les tests unitaires et l'évolution du code.

# 3. Mise en Œuvre sous IntelliJ IDEA

## A. Initialisation et Configuration du Projet

Le projet a été initialisé via **Spring Initializr** (`start.spring.io`), directement intégré dans IntelliJ IDEA. Les dépendances suivantes ont été sélectionnées :

| Dépendance | Artifact Maven | Rôle dans le projet |
|---|---|---|

| | | |
|---|---|---|
| **Spring Web** | `spring-boot-starter-web` | Création de l'API REST (controllers, @RestController) |
| **Spring Data JPA** | `spring-boot-starter-data-jpa` | Abstraction de la persistance via Hibernate / JPA |
| **Lombok** | `lombok` | Réduction du code répétitif (@Getter, @Setter, @Builder) |
| **Validation** | `spring-boot-starter-validation` | Contraintes sur les données (@NotNull, @Min, @Size) |
| **PostgreSQL Driver** | `postgresql` | Pilote JDBC pour connexion à PostgreSQL |

### B. Configuration application.properties

La connexion à PostgreSQL a été configurée dans le fichier `src/main/resources/application.properties` :

### C. Gestion des Dépendances Maven (pom.xml)

Le fichier `pom.xml` a été configuré pour gérer automatiquement les bibliothèques nécessaires. IntelliJ IDEA propose une synchronisation automatique des dépendances Maven, évitant les erreurs de compilation liées aux versions incompatibles.

## 4. Implémentation des Couches

### A. Entité Product (@Entity)

L'entité **Product** est annotée avec `@Entity` et `@Table`. Elle définit le modèle de données persisté dans PostgreSQL .

### B. Repository (Spring Data JPA)

L'interface **ProductRepository** étend `JpaRepository<Product, Long>`. Spring Data JPA génère automatiquement toutes les opérations CRUD sans écriture de SQL manuel .

### C. Service (Logique Métier)

La couche **ProductService** centralise la logique métier et gère les transactions avec `@Transactional` .

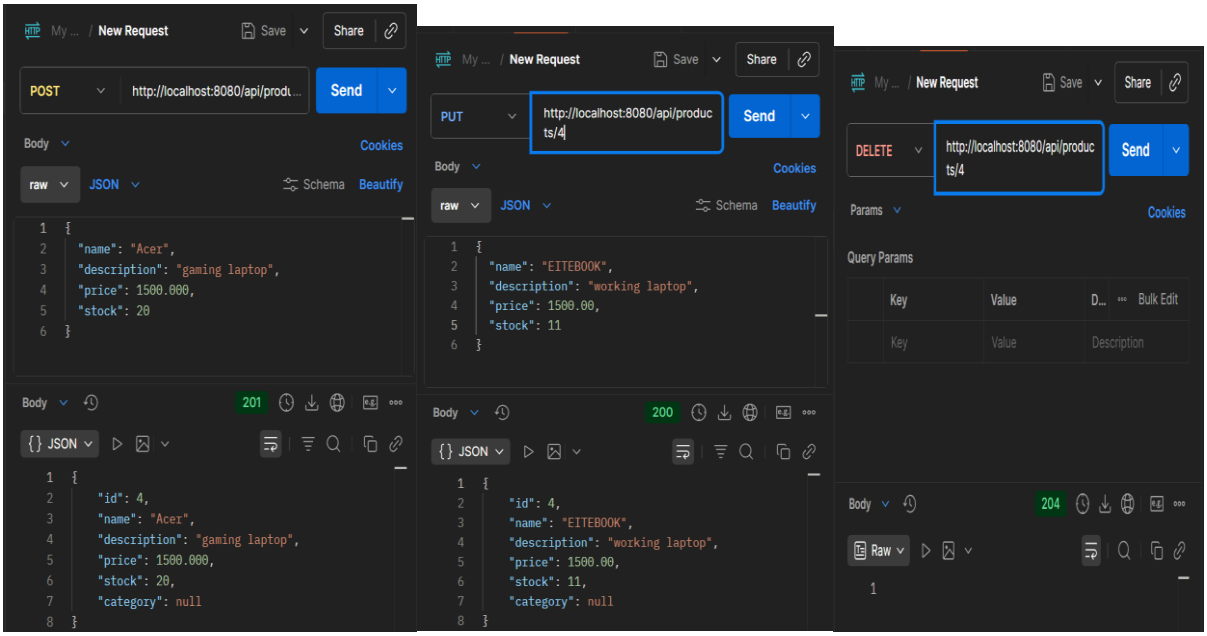
### D. Contrôleur REST (@RestController)

La couche **ProductController** expose les endpoints REST via `@RestController` et `@RequestMapping` :

## 5. Tests et Validation (Postman)

Après le lancement de l'application depuis la console IntelliJ IDEA (`mvn spring-boot:run`), une série de tests a été effectuée avec **Postman** pour valider chaque fonctionnalité CRUD :

| Méthode | URL | Description | Code HTTP |
|---|---|---|---|
| GET | `/api/products` | Récupérer tous les produits | 200 OK |
| GET | `/api/products/{id}` | Récupérer un produit par ID | 200 OK |
| POST | `/api/products` | Créer un nouveau produit | 201 Created |
| PUT | `/api/products/{id}` | Modifier un produit existant | 200 OK |
| DELETE | `/api/products/{id}` | Supprimer un produit | 204 No Content |




## 6. Exercice d'Extension — Relation Category / Product

Conformément aux instructions du TP, l'application a été enrichie avec une nouvelle entité **Category** liée à **Product** par une relation `@ManyToOne`.

### A. Entité Category.

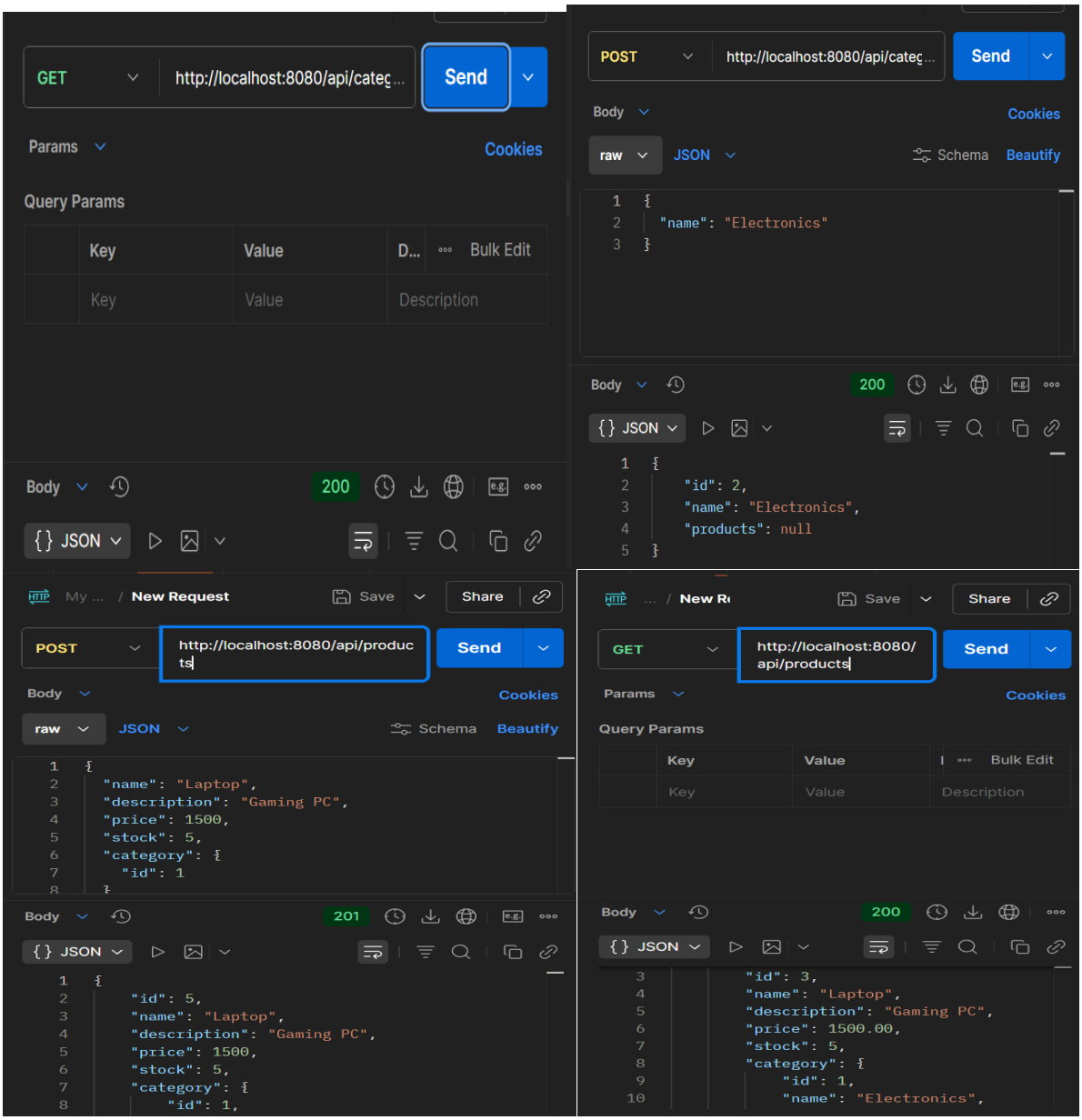
**B.Relation ManyToOne dans Product :** Plusieurs produits peuvent appartenir à une même catégorie.

### C. RepositoryRestResource

L'annotation `@RepositoryRestResource` (Spring Data REST) génère automatiquement une interface HATEOAS complète sans écrire de contrôleur manuel .

| Critère | @RestController manuel | @RepositoryRestResource |
|---|---|---|
| **Code à écrire** | Controller + Service + DTO | Aucun (auto-généré) |
| **Flexibilité** | Totale | Limitée (endpoints standards) |
| **Standard API** | Personnalisable | HATEOAS automatique |
| **Usage recommandé** | API métier complexe | CRUD simple / prototype |
| | | |

## Test avec postman :




## 7. Conclusion

Ce TP a permis de maîtriser le cycle de développement complet d'un **monolithe Spring Boot**, depuis l'initialisation du projet jusqu'à la validation via Postman. L'utilisation d'**IntelliJ IDEA** a considérablement facilité la structuration du code, la gestion des dépendances Maven et le débogage.

| Compétence acquise | Technologie | Résultat |
|---|---|---|
| **Architecture en couches** | `Spring Boot` | Code structuré et maintenable |
| **Persistance ORM** | `Spring Data JPA` | Requêtes SQL générées automatiquement |
| **API REST CRUD** | `@RestController` | 5 endpoints fonctionnels et testés |
| **Validation des données** | `@Valid + @NotBlank` | Rejets automatiques des données invalides |
| **Gestion des transactions** | `@Transactional` | Cohérence des données garantie |
| **API auto-générée** | `@RepositoryRestResource` | Interface HATEOAS sans code manuel |

*Rapport rédigé pour l'architecture monolithe simple — 2026*